Encrypted Chat Application Using AES Encryption

Introduction

This project is a secure client-server chat application that allows multiple users to communicate with each other over a network. The main objective of this project is to ensure secure communication by encrypting messages using AES (Advanced Encryption Standard).

Technologies Used

- Python Programming Language
- Socket Programming (TCP)
- PyCryptodome Library (for AES Encryption)
- Multithreading

Working of the Project

The project consists of two main components:

1. Server:

The server listens for incoming client connections and manages communication between multiple clients. It receives messages from one client and broadcasts them to other connected clients.

2. Client:

Each client connects to the server and can send and receive messages. Before sending, messages are encrypted using AES encryption. Upon receiving, messages are decrypted back to readable text.

Encryption Mechanism

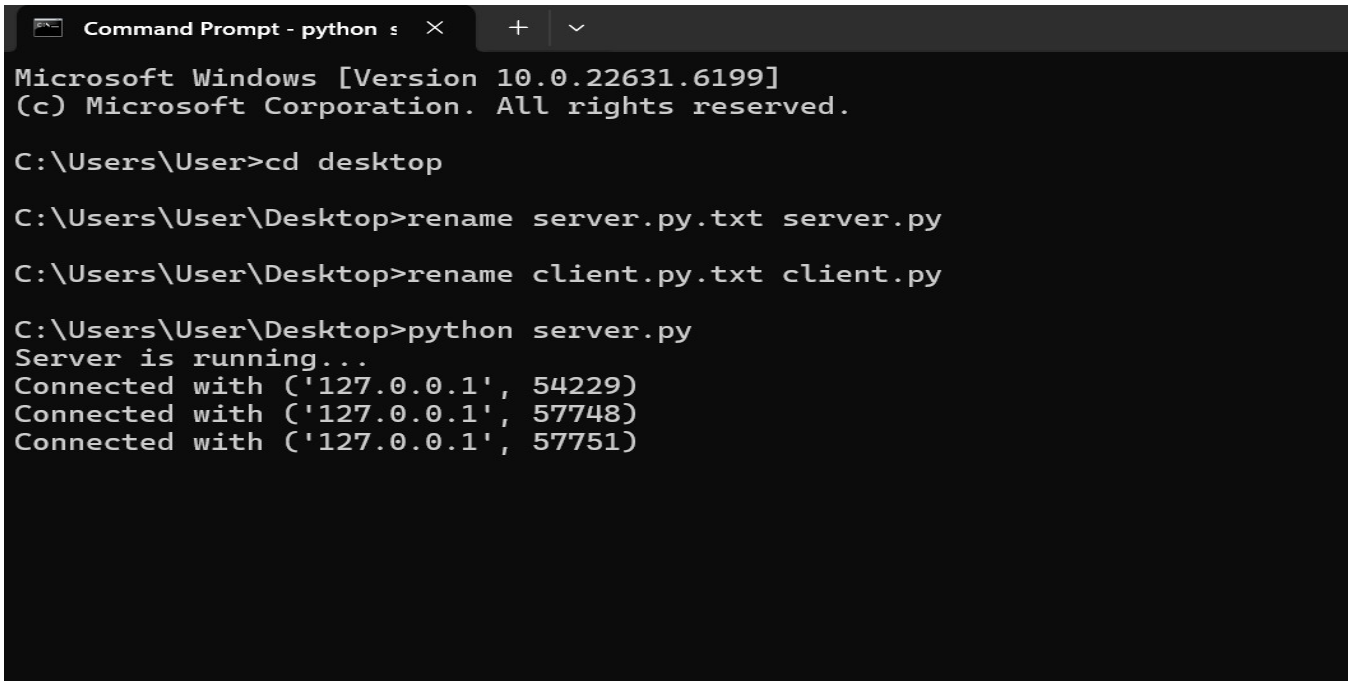
AES (Advanced Encryption Standard) is used for encrypting messages. A pre-shared 16-byte secret key is used for both encryption and decryption. This ensures that the communication remains secure and unreadable to unauthorized users.

Features

- Secure message transmission using AES encryption
- Multiple client support using threading
- Real-time communication
- Simple and efficient implementation

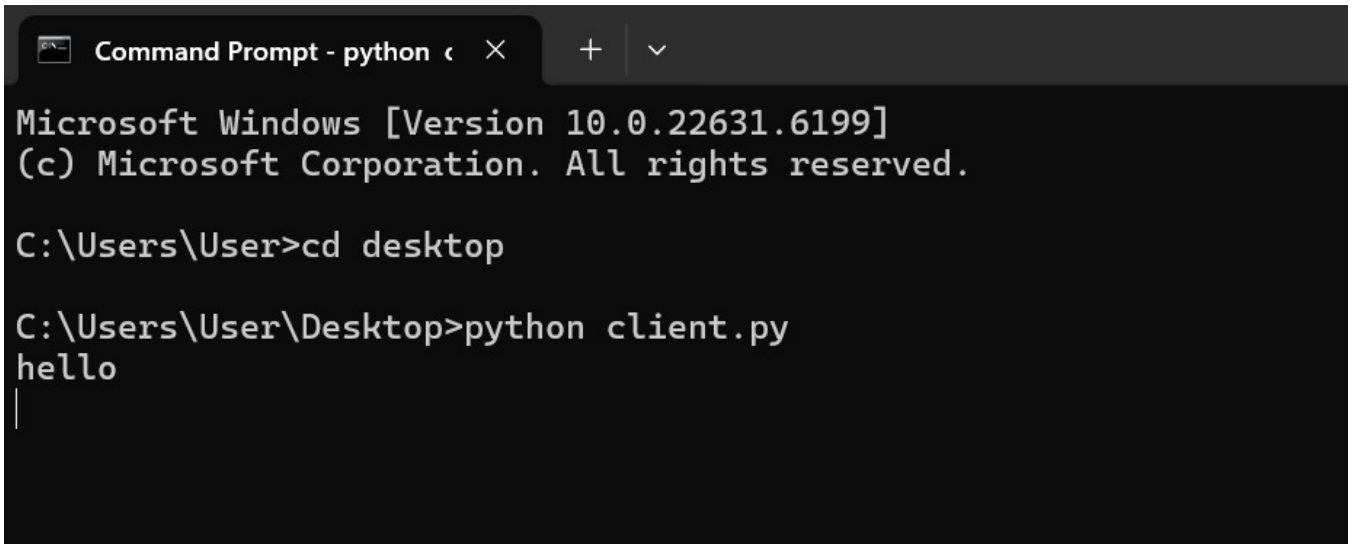
Project Screenshots

Server Running & Client Connection



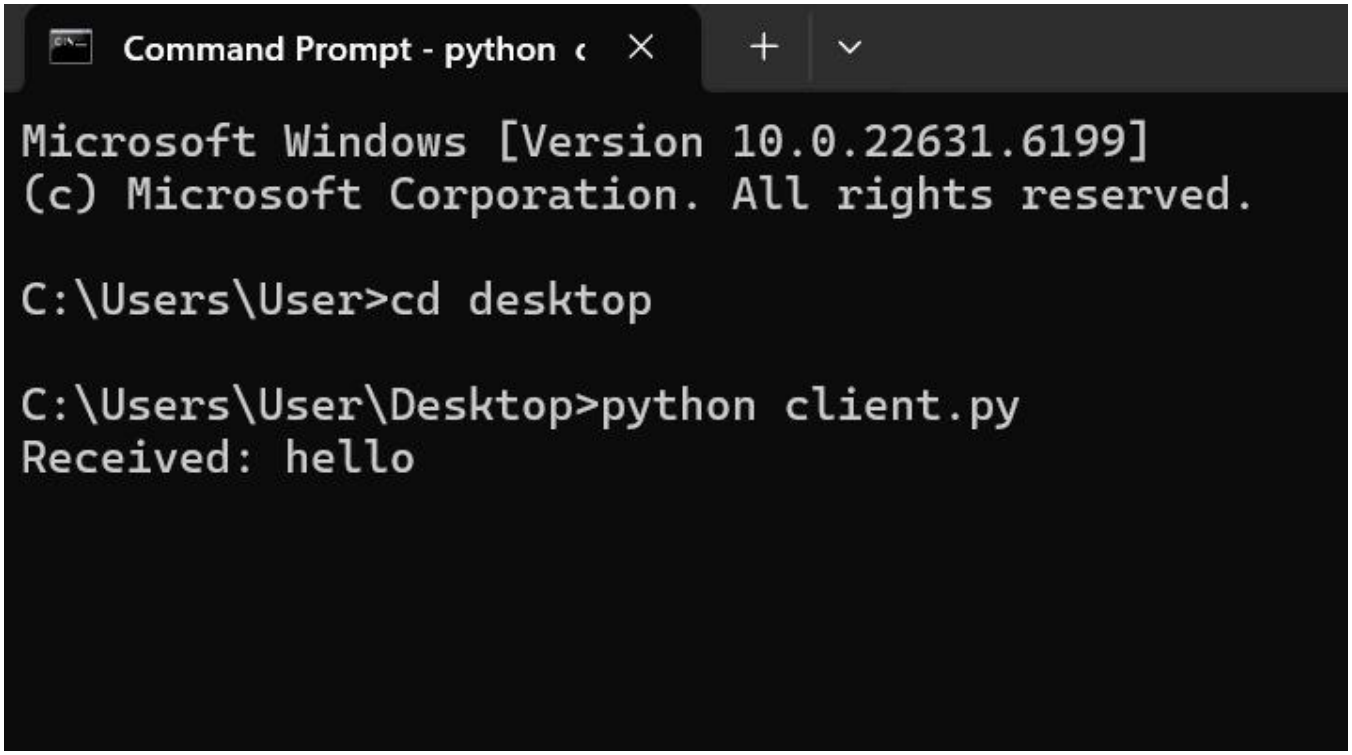
Server running successfully

Message Sending



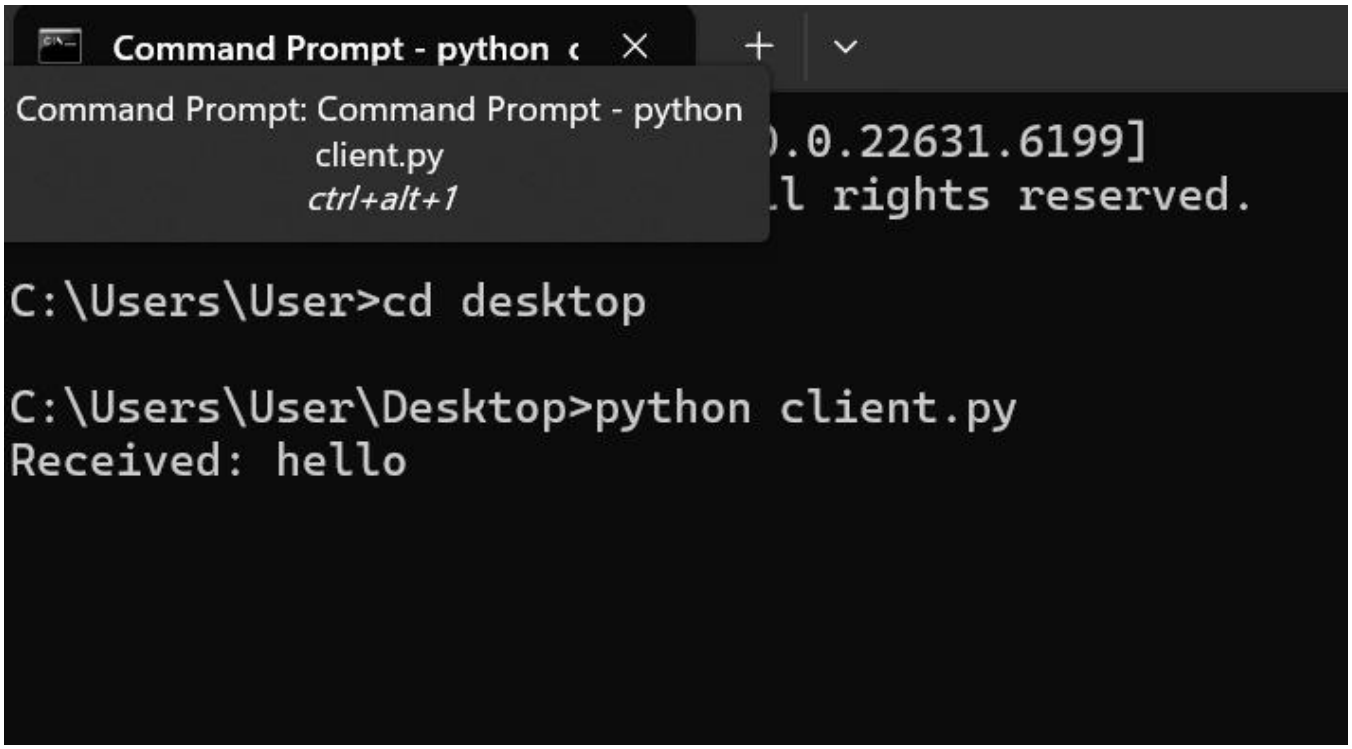
Client sent the message hello

Message Receiving



Client received the message hello

Another Client Received



Client received the message hello

Server.py Code

File Edit Selection View Go Run ··· Search

server.py

C: > Users > User > Desktop > server.py > receive

```python
import socket
import threading

HOST = '127.0.0.1'
PORT = 12345

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()

clients = []

def handle_client(client):
    while True:
        try:
            message = client.recv(1024)
            broadcast(message, client)
        except:
            clients.remove(client)
            client.close()
            break

def broadcast(message, client):
    for c in clients:
        if c != client:
            c.send(message)

def receive():
    print("Server is running...")
```

```python
def receive():
    print("Server is running...")
    while True:
        client, address = server.accept()
        print(f"Connected with {address}")
        clients.append(client)

        thread = threading.Thread(target=handle_client, args=(client,))
        thread.start()

receive()
```

Client.py Code

File Edit Selection View Go Run ··· Search

client.py

C: > Users > User > Desktop > client.py

```python
import socket
import threading
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

HOST = '127.0.0.1'
PORT = 12345

key = b'1234567890123456'  # 16-byte key

def encrypt_message(message):
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.encrypt(pad(message.encode(), 16))

def decrypt_message(message):
    cipher = AES.new(key, AES.MODE_ECB)
    return unpad(cipher.decrypt(message), 16).decode()

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

def receive():
    while True:
        try:
            message = client.recv(1024)
            print("Received:", decrypt_message(message))
        except:
            print("Error!")
            client.close()
```

```python
        except:
            print("Error!")
            client.close()
            break

def write():
    while True:
        msg = input("")
        encrypted = encrypt_message(msg)
        client.send(encrypted)

threading.Thread(target=receive).start()
threading.Thread(target=write).start()
```

Conclusion

This project demonstrates how encryption can be integrated into a chat application to enhance security. It provides a basic understanding of network communication and cryptography concepts in cybersecurity.